

UAS TEKNIK KOMPILASI

Nama : Bayu Setiaji

NIM : 231011403143

Kelas :06TPLM002

1. Pilihan Konstruksi

Konstruksi sintaksis yang dipilih untuk disimulasikan dalam tugas ini adalah **Percabangan (Conditional)**. Untuk mendemonstrasikan pemahaman secara lebih komprehensif, simulasi ini mengimplementasikan *custom syntax* (sintaksis modifikasi), di mana kata kunci standar `if` digantikan dengan `kalo`, dan `else` digantikan dengan `kalogak`.

2. Pembuatan Pattern (Pola Sintaks)

Aturan sintaksis dari konstruksi percabangan modifikasi ini didefinisikan menggunakan pendekatan *Backus-Naur Form* (BNF) sebagai berikut:

```
<kalo_stmt> ::= "kalo" "(" <condition> ")" "{" <statements> "}" "kalogak" "{"  
<statements> "}"  
  
<condition> ::= <identifier> <operator> <value> | <identifier> <operator> <identifier>  
  
<operator> ::= "<" | ">" | "==" | "!=" | "<=" | ">=" |  
  
<statements> ::= <assignment_stmt> | <assignment_stmt> <statements>  
  
<assignment_stmt> ::= <identifier> "=" <expression>  
  
<expression> ::= <identifier> | <value> | <identifier> <arith_op> <value>  
  
<arith_op> ::= "+" | "-" | "*" | "/"
```

3. Implementasi Program

Program simulasi kompilator di bawah ini diimplementasikan menggunakan bahasa pemrograman **Python**. Program ini dirancang untuk merepresentasikan empat tahapan utama dalam proses kompilasi: Analisis Leksikal, Analisis Sintaksis, Analisis Semantik, dan Generasi Kode Antara (*Three-Address Code*).

Source Code Python

```
class KaloCompiler:  
  
    def __init__(self, source_code):  
  
        self.source_code = source_code
```

```

self.label_counter = 1

# Simulasi Tabel Simbol (Symbol Table) untuk tahap Analisis Semantik
# Diasumsikan variabel 'x' dan 'y' telah dideklarasikan sebelumnya dengan tipe integer
self.symbol_table = {'x': 'int', 'y': 'int'}

def new_label(self):
    lbl = f'L{self.label_counter}"
    self.label_counter += 1
    return lbl

# --- TAHAP 1: Analisis Leksikal ---
def lexical_analysis(self):
    # Menambahkan spasi pada karakter khusus untuk memudahkan tokenisasi
    formatted_code = (self.source_code
        .replace('(', ' ( ')
        .replace(')', ' ) ')
        .replace('{', ' { ')
        .replace('}', ' } '))

    # Memecah string menjadi array token
    tokens = formatted_code.split()
    return tokens

# --- TAHAP 2 & 3: Analisis Sintaksis & Semantik ---
def syntax_semantic_analysis(self, tokens):
    try:
        # 2a. Analisis Sintaksis: Memvalidasi keyword awal
        if tokens[0] != 'kalo':
            raise SyntaxError("Sintaksis tidak valid: Konstruksi harus diawali dengan 'kalo'.")

```

```

# Mencari indeks dari setiap batas token struktural

idx_kalo_open = tokens.index('(')
idx_kalo_close = tokens.index(')')
idx_kalo_brace_open = tokens.index('{')
idx_kalo_brace_close = tokens.index('}')

idx_kalogak = tokens.index('kalogak')
idx_kalogak_brace_open = tokens.index('{', idx_kalogak)
idx_kalogak_brace_close = tokens.index('}', idx_kalogak)

# Mengekstrak token berdasarkan blok

condition_tokens = tokens[idx_kalo_open+1 : idx_kalo_close]
kalo_body_tokens = tokens[idx_kalo_brace_open+1 : idx_kalo_brace_close]
kalogak_body_tokens = tokens[idx_kalogak_brace_open+1 :
idx_kalogak_brace_close]

# 3. Analisis Semantik: Validasi keberadaan variabel pada tabel simbol

var_name = condition_tokens[0]
if var_name not in self.symbol_table:
    raise NameError(f"Error Semantik: Variabel '{var_name}' belum dideklarasikan.")

# Membentuk Abstract Syntax Tree (AST) sederhana berbasis dictionary
ast = {
    'type': 'KaloKalogakStatement',
    'condition': " ".join(condition_tokens),
    'kalo_body': " ".join(kalo_body_tokens),
    'kalogak_body': " ".join(kalogak_body_tokens)
}

```

```

        return ast

    except ValueError:

        raise SyntaxError("Format sintaksis tidak lengkap. Harap periksa tanda kurung dan
kurawal.")

# --- TAHAP 4: Generasi Three-Address Code (TAC) ---
def generate_tac(self):

    tokens = self.lexical_analysis()

    ast = self.syntax_semantic_analysis(tokens)

    # Pembuatan label untuk pengaturan alur kontrol (control flow)

    label_else = self.new_label()

    label_end = self.new_label()

    tac = []

    # Evaluasi kondisi: Jika kondisi salah, lompat ke blok kalogak (L1)
    tac.append(f'ifFalse {ast["condition"]} goto {label_else}')

    # Jika kondisi benar, eksekusi blok ini (blok 'kalo')
    tac.append(f'{ast["kalo_body"]}')

    # Setelah blok 'kalo' dieksekusi, lompat ke akhir program (melewati blok 'kalogak')
    tac.append(f'goto {label_end}')

    # Penanda awal blok 'kalogak'
    tac.append(f'{label_else}:')

    tac.append(f'{ast["kalogak_body"]}')

```

```

# Penanda akhir konstruksi
tac.append(f'{label_end}:')

return {
    'tokens': tokens,
    'ast': ast,
    'tac': "\n".join(tac)
}

# =====
# CONTOH PENGGUNAAN PROGRAM
# =====

if __name__ == "__main__":
    # Source code yang akan diproses
    source = "kalo ( x > 5 ) { y = 1 } kalogak { y = 0 }"

    compiler = KaloCompiler(source)
    result = compiler.generate_tac()

    print("--- 1. Hasil Analisis Leksikal (Tokens) ---")
    print(result['tokens'], "\n")

    print("--- 2 & 3. Hasil Sintaksis & Semantik (AST) ---")
    print(result['ast'], "\n")

    print("--- 4. Generasi Three-Address Code (TAC) ---")
    print(result['tac'])

```

4. Penjelasan dan Dokumentasi Tahapan

Implementasi program di atas mendemonstrasikan proses kompilasi kode sumber (*source code*) melalui empat tahapan utama, dengan penjelasan sebagai berikut:

A. Analisis Leksikal (*Scanner / Lexer*)

Pada metode `lexical_analysis()`, program bertugas memindai karakter dari *source code* dan memecahnya menjadi sekumpulan leksik atau token. Program menyisipkan spasi tambahan pada karakter pembatas struktural (seperti `(, ), {, dan }`) agar metode pemisahan teks dapat mengidentifikasi *keyword*, operator, pengenalan (*identifier*), dan tanda baca sebagai token tunggal yang valid.

- **Input:** `"kalo ( x > 5 ) { y = 1 } kalogak { y = 0 }"`
- **Output (Array Token):** `['kalo', '(', 'x', '>', '5', ')', '{', 'y', '=', '1', '}', 'kalogak', '{', 'y', '=', '0', '}' ]`

B. Analisis Sintaksis (*Parser*)

Tahapan ini dieksekusi dalam metode `syntax_semantic_analysis()`. Urutan token yang dihasilkan dari tahap leksikal dievaluasi berdasarkan aturan tata bahasa (BNF) yang telah ditetapkan. Program mencari indeks token penanda blok untuk mengekstrak tiga elemen utama: kondisi logika, sekumpulan pernyataan pada blok `kalo`, dan sekumpulan pernyataan pada blok `kalogak`. Elemen-elemen tersebut kemudian direpresentasikan secara hierarkis dalam bentuk *Abstract Syntax Tree* (AST) sederhana. Apabila susunan token tidak sesuai (misal: kurung kurawal tidak lengkap atau *keyword* salah), kompilator akan menghasilkan pesan *Syntax Error*.

C. Analisis Semantik

Analisis semantik dilakukan secara simultan bersamaan dengan analisis sintaksis untuk memvalidasi kesahihan makna program. Pada simulasi ini, kompilator melakukan pengecekan lingkup variabel (*scope checking*). Program memeriksa apakah operan pertama di dalam kondisi (yaitu variabel `x`) sudah terdaftar di dalam struktur data Tabel Simbol (`self.symbol_table`). Jika pengembang menggunakan variabel yang belum dideklarasikan, program secara otomatis akan menghentikan proses kompilasi dan memberikan peringatan *Name Error* atau *Semantic Error*.

D. Generasi Kode Antara / *Three-Address Code* (TAC)

Pada metode `generate_tac()`, struktur AST ditranslasikan menjadi kode penengah (TAC). TAC merepresentasikan alur kontrol program dalam bentuk instruksi sekuensial tingkat rendah menggunakan mekanisme pelabelan (*label*) dan lompatan bersyarat maupun tak bersyarat (*jump / goto*).

1. **ifFalse x > 5 goto L1:** Instruksi lompatan bersyarat. Jika hasil evaluasi kondisi bernilai salah (*false*), alur eksekusi akan diarahkan langsung ke Label 1 (L1) yang merupakan blok `kalogak`.
2. **y = 1:** Instruksi penugasan (dieksekusi apabila kondisi bernilai benar).

3. **goto L2:** Instruksi lompatan tanpa syarat. Memaksa program untuk melewati blok kalogak dan langsung menuju Label 2 (L2) sebagai tanda akhir konstruksi.